czii-cryo-et-object-identification

9位解法

Rank	9
Team	Avengers
Author	NANACHI
Topic ID	561431
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561431
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

まず、このような魅力的なコンペを開催してくださったコンペ主催者と Kaggle スタッフに心から感謝します。このコンペを存分に楽しみ、その過程で多くを学びました！

また、@hengck23 と @davidlist にも感謝します。@hengck23 の discussion と notebook は私の解法の出発点となり、@davidlist の多くの discussion とコメントは非常に役立ちました。

概要

私の解法はシンプルで素直なものです。3D の ConvNeXt 風のモデルでセグメンテーションを行い、さらに、できるだけ多くのモデルのアンサンブルを使いました。その後、cc3d を使って粒子の重心を計算し、予測した重心を DBSCAN で整理しました。

セグメンテーションのマスク

各粒子について、半径を調整した正解マスクを使いました。その結果、public leaderboard のスコアが 0.02~0.04 跳ね上がりました。具体的には、次の表のようにマスクのサイズを調整しています。

apo-ferritin(r=60)	beta-galactosidase(r=90)	ribosome(r=150)	thyroglobulin(r=130)	virus-like-particle(r=135)
60/2	90/2	150/3	130/3	135/3

コンペの指標では、予測した重心が正解の重心から r×0.5 の半径以内に入ることが求められるので、各粒子の半径に 0.5 以下の係数を掛けるのは理にかなっていると考えています。

モデルのアーキテクチャ

この節では、エンコーダとデコーダの両方について、私のモデルのアーキテクチャを説明します。

エンコーダ

エンコーダの実装には ConvNeXt から始めました。ConvNeXt はとても強力で高速だからです。そして、小さな粒子を予測するというタスクに合うようにモデルをカスタマイズしました。元の ConvNeXt からの変更点は次のとおりです。

- すべての畳み込み層を 2D から 3D に
- 4x4 の stem -> 2x2 の stem。粒子はとても小さいので、大きな stem はモデルの予測、特に apo-ferritin と beta-galactosidase に悪影響を与えると考えました。この変更でモデルの推論は遅くなりましたが、交差検証の性能は改善しました。
- conv block のカーネルサイズを 7x7 ->3x3 に。この変更の理由は stem と同じです。この変更で推論速度と cv の両方が改善しました。
- ブロック数を (3, 3, 9, 3) -> (3, 3, 3, 3) に。この変更は、パラメータ数を減らし、推論速度を上げるためのものです。cv は下がりませんでした。

デコーダ

デコーダの流れは U-Net に基づいています。conv block の実装を次のコードに示します。

```
 (nn.Module):
   ():
         ().__init__()
         .conv1 = nn.Conv3d(in_channels, in_channels, kernel_size=, padding=, bias=,
groups=in_channels)
         .norm1 = nn.GroupNorm(num_groups=, num_channels=in_channels)
         .act1 = nn.GELU()
         .conv2 = nn.Conv3d(in_channels, in_channels*, kernel_size=, bias=)
         .conv3 = nn.Conv3d(in_channels*, out_channels, kernel_size=, bias=)

   ():

         x = .conv1(x)
         x = .norm1(x)
         x = .conv2(x)
         x = .act1(x)
         x = .conv3(x)
          x
```

デコーダの実装を次のコードに示します。

```
 (nn.Module):
   ():
         ().__init__()
         .up3 = nn.Upsample(scale_factor=(,,), mode=, align_corners=)

         .dec3 = ConvBlock3D(in_channels=encoder_dims[] + encoder_dims[], out_channels=decoder_dims[])
     …

   ():
         x, f0, f1, f2, f3 = features


         d3 = .up3(f3)
         d3 = torch.cat([d3, f2], dim=)
         d3 = .dec3(d3)
     …
```

セグメンテーションの正解は単なる球なので、パラメータを減らすため、アップサンプリングには nn.ConvTranspose3d ではなく基本的な nn.Upsample を使いました。

その他の詳細

- loss 関数は bce
- 私の推論コードは @hengck の この notebook に基づいており、DBScan のアイデアは @linheshen の notebook から得ました。
- 推論のウィンドウサイズは (32, 320, 320) です（z 軸のオーバーラップは 6）。学習のウィンドウサイズは (32, 256, 256) です。
- TTA は rot90、180、270 です。
- 前処理は正規化だけです

```
():
      lower, upper = np.percentile(x, (, ))
      x = np.clip(x, lower, upper)
      x = x - np.(x)
      x = x / np.(x)
       x
```

- 学習のデータ拡張は rot90、180、270 と xyz の反転です。

効かなかったこと

- 主催者が提供した合成データでの事前学習
- マスクをさらに小さくする（例: 大きな粒子で係数 0.25、小さな粒子で係数 0.4）
- rot90 の代わりに flip の tta

コードとモデルのチェックポイント

推論コード: https://www.kaggle.com/code/wadakoki/czii-9th-inference

学習コード: https://colab.research.google.com/drive/1OG5Rap9zUkKG6fnGZ8kP0JwwFZfSTgCc?usp=sharing

モデルのチェックポイント: https://www.kaggle.com/datasets/wadakoki/czii-9th-final-models/data

生成したマスク: https://www.kaggle.com/datasets/wadakoki/czii-mask-small

マスク生成のコード: https://www.kaggle.com/code/wadakoki/czii-gen-small-masks

google colab での学習コードの要件

- GPU: L4
- RAM: High Memory